

CS4031--Compiler Construction

Course Project - Spring 2026

Total Marks: 100 (Weightage 10%)

Max. 3 Members allowed in a group.

Final Submission Deadline: 10th May 2026

Semester Project: Retro Computing Mini Compiler -- Creative Design Challenge

Objective:

To design and implement a compiler for a *retro-inspired mini language* that translates source code into a target format of your choice. The project must demonstrate comprehensive understanding of all compiler phases through both documented theory and working implementation, with emphasis on creative language design.

Project Requirements

1. Design Your Own Retro-Style Language

Each group will create and document a small programming language inspired by vintage computing concepts or specialized domains. Choose one of the following themes:

- **8-Bit Adventure:** A language for describing text-based adventure games (rooms, items, player actions, puzzles)
- **Chip-8 Inspired:** A high-level language that compiles to Chip-8 assembly or similar retro platform
- **Turtle Graphics DSL:** A LOGO-inspired language for generating geometric patterns and animations
- **Retro Music Composer:** A language for generating chiptune-style music (notes, patterns, effects)
- **Pixel Art Generator:** A scripting language for creating pixel art patterns and transformations
- **Old-School Calculator:** A language for programmable calculators with functions and variables
- **Choose Your Own Adventure:** A language for creating interactive fiction with branching narratives
- **Retro CPU Simulator:** A language that compiles to a simple custom assembly for a simulated CPU

Each team must **write their complete language specification**, including:

- **Syntax specification** using EBNF/CFG notation with clear non-terminals < >
- **Lexical grammar** (tokens, keywords, operators, identifiers, literals)
- **Type system** and semantic rules (if applicable)

- **Memory model** (variables, scope, allocation strategy)
- **At least 3 complete example programs** with expected inputs and outputs
- **Error handling strategy** (syntax and semantic error reporting)

2. Implement All Compiler Phases with Documentation

Students must demonstrate a **complete compilation pipeline** with proper documentation for each phase:

Phase	Requirements	Deliverable
1. Lexical Analysis	Define token categories using regular expressions; design DFA/NFA	Hand-drawn state transition diagram or transition table
2. Syntax Analysis	Write complete grammar; handle left recursion and common prefixes if present	At least 2 handwritten parse trees for sample inputs
3. Semantic Analysis	Symbol table design; type checking; scope management	Handwritten symbol table example with scope visualization
4. Intermediate Code	Generate Three-Address Code (TAC), abstract syntax tree, or custom IR	Printed IR examples for test cases
5. Basic Optimization	Implement at least 2 optimizations (constant folding, dead code elimination, peephole optimization)	Before/after optimization examples
6. Code Generation	Generate executable output or interpret the IR with a virtual machine	Output demonstration for all test cases

3. Technical Implementation Requirements

- **Programming Language:** Python (recommended), Java, C++, or Rust
- **Interface:** Command-line interface with the following features:

`compiler.exe input.src -o output`

`compiler.exe input.src --debug` (shows intermediate representations)

`compiler.exe --interactive` (REPL mode for testing snippets)

- **Input:** Source files with custom extension (e.g., .adv, .chip, .logo)
- **Output:** Depending on your language design:
 - Executable output (assembly, bytecode, or interpreted result)
 - Generated output file (image, sound, game data, etc.)
 - Direct execution results displayed to user
- **Error Reporting:** Meaningful error messages with line/column information

4. Project Milestones & Timeline

Milestone	Due Date	Description
Proposal Submission	27th March 2026	Language concept, team members, basic syntax examples
Phase 1-2 Complete	10th April 2026	Lexical and syntax analysis working; handwritten docs submitted
Phase 3-4 Complete	24th April 2026	Semantic analysis and IR generation working
Full Implementation	8th May 2026	Complete compiler with optimizations and code generation
Final Submission & Demo	11th to 15th May 2026	All deliverables + viva demonstration

5. Final Submission Deliverables

Each group must submit the following items in a single ZIP file or Git repository:

A. Handwritten Documentation (Scanned)

1. **Lexical Analyzer Design:** DFA diagram or transition table for token recognition
2. **Parser Design:** Two complete parse trees for different input programs
3. **Semantic Analyzer Design:** Filled symbol table showing scopes and type information

B. Digital Documentation

4. **Language Reference Manual:** Complete specification of your mini language
5. **Compiler Architecture Document:** Overview of implementation with phase-wise breakdown
6. **Test Suite:** At least 5 test programs with expected outputs
7. **Team Reflection (2 pages):**
 - Challenges faced and solutions
 - What you learned about compiler design
 - Future improvements if you had more time
 - Individual contributions of each team member

C. Implementation

8. **Complete Source Code** with:
 - Clear comments explaining each compiler phase
 - README file with setup and usage instructions
 - Git repository URL with meaningful commit history

D. Demonstration

9. **Live Viva Demonstration** (5-10 minutes per group):
- Show compiler working on 3 distinct test cases
 - Demonstrate error handling
 - Explain design decisions
 - Answer questions about implementation

6. Grading Rubric

Component	Marks	Criteria
Language Design	10	Creativity, completeness, clarity of specification
Lexical Analysis	10	Correct tokenization, DFA design, error handling
Syntax Analysis	10	Grammar correctness, parse tree generation, error recovery
Semantic Analysis	10	Symbol table, type checking, scope management
IR Generation	10	Quality of intermediate representation
Optimization	10	Working optimizations with clear demonstration
Code Generation	10	Correct output generation for target format
Documentation	10	Handwritten artifacts, language manual, reflection
Demo & Viva	20	Successful demonstration, understanding of concepts